

SIGPROC

SIGPROC is a package designed to standardize the initial analysis of the many types of fast-sampled pulsar and FRB data. Installation instructions can be found at their website (<http://sigproc.sourceforge.net/>) along with documentation on how to use it (<http://sigproc.sourceforge.net/sigproc.pdf>). The documentation offers information on all of SIGPROC's programs, though this information is sometimes not sufficient. This paper discusses the programs that are likely to be used most often in greater detail, along with some other general information.

In order for SIGPROC to analyze data, the data must be in a filterbank format file. The data contained within a filterbank file is a 2D array of intensity values over a range of frequency channels and time samples. This data can contain signals from FRBs. Filterbank files are separated into two sections: the header and the body. The header contains all relevant information about the data such as sampling frequency, frequency channel bandwidth, and the number of frequency channels. The body contains the 2D array, however, this array is formatted as a 1D array. The ordering of the elements in the 1D array is extremely important, and it is covered quite extensively in the SIGPROC documentation so I won't go into it here. I would just like to mention that the frequency channels should be ordered from highest to lowest. Filterbank files must have the extension “.fil”.

To use SIGPROC programs, use this format:

```
% (program name) (input file(if needed)) (arguments) > (output file name)
```

There are examples of this throughout this paper. For quick access to information about a program and the arguments it takes, just enter the program name followed by “help”:

```
% (program name) help
```

Fake

Fake creates filterbank files containing signals of fake pulsars. You can specify the properties of the pulsar such as period, observation time, and dispersion measure. Fake is not set up to create simulated data from FRBs, however, it is possible to “cheat” and make it do so anyway. To make fake simulate FRBs instead of pulsars, just make the pulse period extremely large, at least as long as observation time. If you make the pulse period equal to the observation time, the FRB signal at the highest frequency channel will occur at half the observation time. When doing this, be careful when setting the pulse width because the width can only be set to be a certain percentage of the period, it can not be set to a specific value. Fake is one of the few SIGPROC programs to not require an input file. Here is an example of how to use fake:

```
% fake -period 200000 -width 0.025 -snrpeak 6.0 -dm 500 -nbits 16 -nchans 200 -tsamp 400 -tobs 200  
-fch1 800 -fch2 2 > fake16.fil
```

This will create a filterbank file of the name “fake16.fil” that contains a simulated FRB signal. The FRB has a DM of 500 and width of 0.05 s. There are 200 frequency channels spaced 2 MHz apart starting at 800 Mhz. The sampling frequency is 400 μ s, the SNR of the dedispersed pulse would be 6.0, and the values of the array are 16 bit numbers.

Dedisperse

As the name suggests, Dedisperse dedisperses a time series. It sums the signals from all the frequency channels together. The time offset of the samples being summed is dependent on the dispersion measure. Dedisperse requires a filterbank file as input. The extension of the output file should be “.tim”. Here is an example:

```
% dedisperse fake16.fil -d 500 > dedisp16.fil
```

This will dedisperse fake16.fil at a DM value of 500 and output the resulting 1D time series in a file named dedisp16.tim. .tim files are similar to filterbank files, except there is a reference DM in the header and the array in the body is a true 1D array.

Reader

Reader converts non-human-readable file formats like .tim and .fil into a human-readable format. The output file of reader should have the extension “.ascii”. Here is an example:

```
% reader fake16.fil -c 1 -c 2 -c 3 > read16_1.ascii
```

This writes the intensity values from the first three frequency channels to read16.ascii. Without the -c arguments, data from all frequency channels would be written. read16.fil should look something like this:

0.000000	164.000000	192.000000	158.000000
0.000400	126.000000	144.000000	125.000000
0.000800	187.000000	218.000000	226.000000
...	...	...	...

The first column lists the times of the samples, and the 2nd, 3rd, and 4th columns list the intensity values from the 1st, 2nd, and 3rd frequency channels respectively. Here is another example:

```
% reader dedisp16.tim > read16_2.ascii
```

This will produce a similar output file, but this one will only have one intensity column since

dedisp16.tim contains a 1D dedispersed time series. I wrote up a quick program called "homemade_plotter.py" that plots data in .ascii files, though it can only plot dedispersed time series or data from a single frequency channel.

It may be useful to know that some of SIGPROC's programs can be piped together. This eliminates the need of some temporary files. For example:

```
% fake -period 200000 -width 0.025 -snrpeak 6.0 -dm 500 -nbits 16 -nchans 200 -tsamp 400 -tobs 200  
-fch1 800 -fch2 2 | dedisperse -d 500 | reader > read16_3.ascii
```

If the data from read16_2.ascii and read16_3.ascii are plotted, the plots should be nearly identical visually, the only difference coming from the random noise.

Seek

Seek looks through a dedispersed time series and searches for pulsars by default, however, it can be set to search for single pulses instead.

```
% seek dedisp16.tim -nofft -pulse
```

The arguments "-nofft" and "-pulse" set seek to look for FRBs in dedisp16.tim. The file extension of the input file must be ".tim". When seek is set to search for pulses, the output file will have the same name as the input file, except the file extension will be ".pls". In the case above, the output file would be "dedisp16.pls". ".pls" files look something like this:

500.000	7	210917	12.24	19
500.000	7	210981	11.53	19
500.000	6	210972	11.27	19
...	...	...	...	...

There are five columns. The first column is the dispersion measure of the pulse. The second column is n , where the pulse width is $\text{tsamp} \times 2^n$. The third column is n_{sample} , where $n_{\text{sample}} \times \text{tsamp}$ gives the time at which the pulse was found relative to the start of the observation. The fourth column is the SNR of the pulse. The fifth column is m , where 2^m gives the nearest power of 2 for a FFT of the time series, though this information is rarely used.

On SIGPROC's website, there is a page that discusses seek when it is set to search for pulsars (<http://sigproc.sourceforge.net/seek/>), so won't talk about it much here. However, I would just like to point out that if seek is set to search for pulsars, the output file will have the same name as the input file, except the file extension will be ".prd".

Hunt

Hunt is very similar to seek, except it allows FRBs and pulsars to be searched for over a range of DM values. The DM values to be tested must be listed in a file named “dmlist”. An easy way to create this file is to do the following:

```
% step (DM_start) (DM_stop) (DM_step) > dmlist          <-- General Form
```

```
% step 450 550 0.5 > dmlist                          <-- Specific Case
```

The specific case above will create a file named “dmlist” which lists numbers from 450 to 550 in steps of 0.5.

Since hunt searches over a range of DM values, the input file can not be a dedispersed time series; it must be a filterbank file (have the extension “.fil”). Just like seek, hunt will search for pulsars by default, but it can be set to search for single pulses by using the same arguments. For example:

```
% hunt fake16 -nofft -pulse
```

Note that you need not state the file extension of the input file. Hunt knows to use fake16.fil. Hunt essentially runs dedisperse and seek in a loop. For each DM value listed in dmlist, it will dedisperse fake16.fil at the DM value and run seek with the resulting .tim file. All FRBs found will be appended to a single .pls file, and just like seek, the output file will have the same name as the input file, except the file extension will be “.pls”.